3D Radio Environment Map Modeling for Aerial Wireless Networks

Ryan Frost
Advisor: Dr. Liu
Collaborators: Shadab & Dr. Haihan

VIRGINIA TECH

Wireless @ Virginia Tech

Background

- M.Eng. in CPE Candidate
- B.S. in Engineering from JMU
- **Interest Areas:**
  - Embedded Systems
  - Cyber-Physical Systems
  - Edge Machine Learning

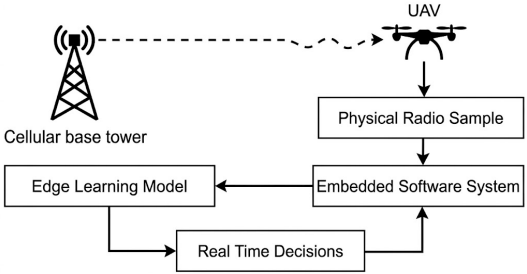
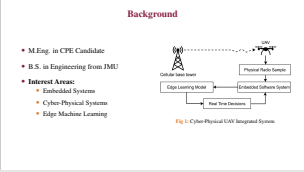


Fig 1: Cyber-Physical UAV Integrated System



Here is a quick background on me. I'm a Masters of Engineering in Computer Engineering Candidate here at Virginia Tech. I have a Bachelor's of Science in Engineering from JMU. My intereset areas are embedded systems, cyber-physical systems, and edge machine learning. I'm interested not only in where physics meets software, but also how we can leverage light weight learning models to make decisions on those physical measurements without access to unlimited compute resources. This led me to pursue Dr. Liu's research team where I've had the opportunity to research the role Unmanned Aerial Vehicles (UAVs) play in quickly deploying wireless networks in unique scenarios.

Research Space

- Why are UAVs being used for wireless networks?
- **Scenarios:**
  - Disaster relief
  - Remote areas
  - Tactical operations

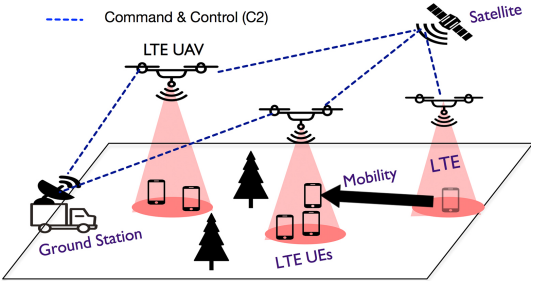
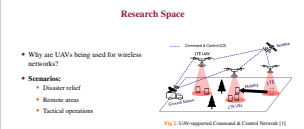


Fig 2: UAV-supported Command & Control Network [1]



To set the foundation of the project, it is important to understand why UAVs are being used for wireless networks. As it turns out, they are very useful for rapidly deploying aerial base stations when the usual grounded base station infrastructure is either destroyed or non existent. Example scenarios include disaster relief, remote areas of the planet, and tactical operations. Advances in UAV technology has allowed for this innovation in wireless networks, and this is a wide area of research sponsored by private companies and the government alike.

Problem Space

- How do we provide connection to the widest area using UAVs?
- **Challenges:**
  - Varying altitude adds new dimension
  - Limited sample size
  - Limited power and compute on UAV

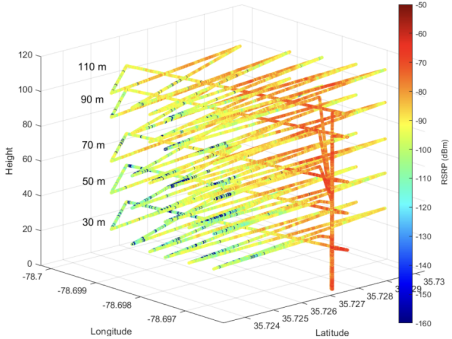
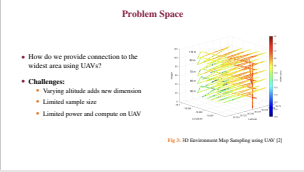


Fig 3: 3D Environment Map Sampling using UAV [2]



The natural next question to ask when setting up a UAV-based wireless network is how do we provide connection to the widest area with the resources we have? This question poses its own unique challenges for aerial networks beyond the established ground-based networks. The most obvious challenge is the varying altitude of UAVs. We now want to understand connectivity in 3 dimesions to know where to position our UAVs. Not only do we add a new dimension, but based on the scenarios described earlier, we are working in areas that have little to no radio map data. Disasters will change the connectivity landscape, and remote areas and tactical operations will inherintly have little to no prior data. That is on top of collecting samples in a whole new dimension, making samples even more sparse. Additionally, we don't have the power supply of a ground base station, which means limited real-time compute on the UAV itself. This is the problem space the my project works in.

Project Layout

- 5 Month Timeframe
- **Phases:**
  1. Analyze existing AERPAW datasets
  2. Develop dataset processing pipeline
  3. Develop machine learning pipeline
  4. Explore extreme learning machine

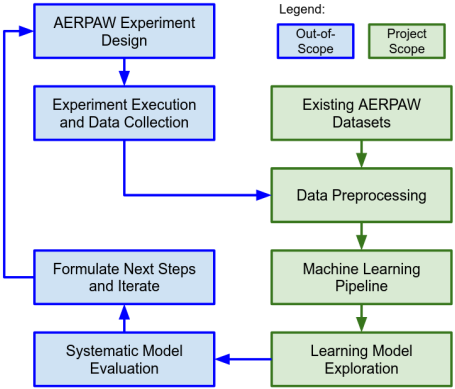


Fig 4: Project Layout Flowchart

Project Layout

- 5 Month Timeframe
- **Phases:**
  1. Analyze existing AERPAW datasets
  2. Develop dataset processing pipeline
  3. Develop machine learning pipeline
  4. Explore extreme learning machine

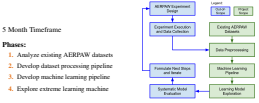


Fig 4: Project Layout Flowchart

Given the problem space, I worked with Dr. Liu's student Shadab as well as Dr. Haihan to establish a 5 month project that would be both a standalone exploration of the problem space and serve as a starting point for future research. Our starting point was NC State's Aerial Experimentation and Research Platform for Advanced Wireless, hence known as AERPAW. They have a number of published datasets from UAV flights with RSRP samples and a single base station. For this project, we downselected the published datasets and selected a few to work with. Then I worked on a dataset processing pipeline to evaluate the suitability of selected datasets. I also created a machine learning pipeline that builds on the processing pipeline to feed the data into learning models. Finally I explored the viability of an extreme learning model compared to baseline models for in-flight real-time training and inference.

Phase 1: AERPAW Dataset Analysis

- AERPAW vs Digital Twin
- **Dataset Criteria:**
  - UAV-based
  - 3D trajectory
  - RSRP values

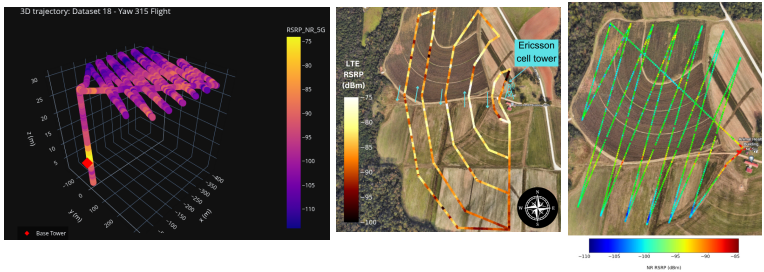
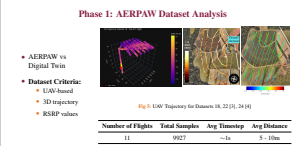


Fig 5: UAV Trajectory for Datasets 18, 22 [3], 24 [4]

Number of Flights	Total Samples	Avg Timestep	Avg Distance
11	9927	∼1s	5 - 10m



To briefly explain AERPAW, AERPAW is a platform where a researcher can program an experiment virtually and schedule it to be run on NC State's drone field testbed. This is very useful for real world, repeatable data collection. But why would we need real world data when there are already existing digital twin-based simulated radio environment maps? Well, simluated environments lack the nuances and fluctuations that exist in real-world scenarios, and those fluctuations act as crucial features for a learning model to train on. I then had to analyze the 28 published datasets based on three criteria: the dataset had to be UAV-based with a 3D trajectory and recorded RSRP values. Based on this, the team selected datasets 18, 22, and 24 for further analysis.

Phase 2: Data Processing Pipeline

- Medain Absolute Deviation (MAD)
- Spatial Correlation
- Fast-Fading Correlation

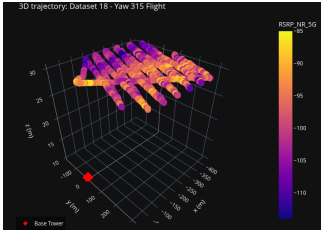
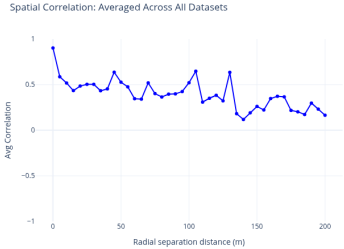
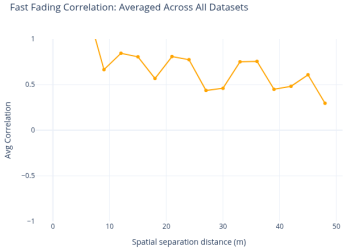
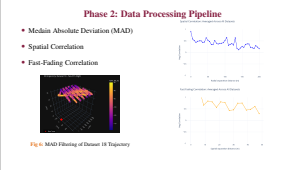


Fig 6: MAD Filtering of Dataset 18 Trajectory







The first step in using the datasets was converting them all to the same format. One useful processing technique was taking the medain altitude to remove the UAV ascent and descent. This was done to focus on the variation in RSRP at further distances. To better understand how RSRP values change with space and time, a spatial correaltion and fast-fading correlation analyses were performed. At this point, we were mimicing some very recent work by folks at NC State (Gautham Reddy, Kursat Tekbıyık, Bryton Petersen, Antoine Lesage-Landry, Gunes Karabulut Kurt, and Ismail Guvenc). The spatial correlation shows how datapoints that are close in space have higher correlation within the same region. The fast-fading correlation shows how datapoints that are close in both space and time have higher correlation, although the sample-to-sample RSRP has greater variation than the region-based analysis likely due to to changing line of sight conditions. Essentially, we are identifiying the opportuinity for a learning model to actually learn.

Phase 3: Machine Learning Pipeline

- Line of Sight Path Loss
  - RSME: 17.4918 dB
- PyTorch Dataloader
- InceptionTime

$$FSPL = \left(\frac{4\pi df}{c}\right)^2$$

Eq. 1: Line of Sight Path Loss
(d = distance, f = frequency, c = speed of light)

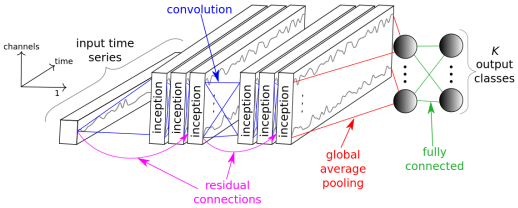


Fig 7: Inception Network for Time Series Classification [5]

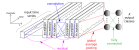
Train %	Val %	Test %	Epochs	Test RSME	Train Time
0.7	0.15	0.15	100	3.6468 dB	577.36s

Phase 3: Machine Learning Pipeline

- Line of Sight Path Loss
  - RSME: 17.4918 dB
- PyTorch Dataloader
- InceptionTime

$$FSPL = \left(\frac{4\pi d f}{c}\right)^2$$

Eq. 1: Line of Sight Path Loss
(d = distance, f = frequency, c = speed of light)


input sequence
Conv
1D Pooling
Conv
1D Pooling
Conv
1D Pooling
Conv
1D Pooling
Global Average Pooling
Fully connected
Global Average pooling
Fully connected
Y
Y
Y

Phase 4: Extreme Learning Machine

- Limited Sample Analysis
- Limited Compute Analysis

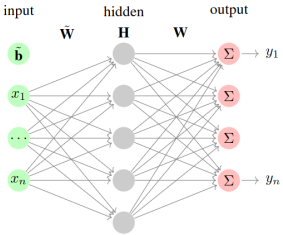


Fig 8: Extreme Learning Machine Model [6]

$$f(x) = \sum_{i=1}^{L} \beta_i g(w_i \cdot x + b_i)$$

Eq. 2: General single-hidden layer feedfoward network (SLFN) equation

w_i = wieght vector input to node i
b_i = bias of node i
β_i = trained ouptut weight

$$\hat{\beta} = H^{\dagger} T$$

Eq. 3: Moore-Penrose generalized inverse optimization

Samples	Inception RSME	ELM RSME	Inception Train Time	ELM Fit Time	Inception Epochs
100	4.1208 dB	7.7501 dB	66.25s	0.0950s	38
500	3.7646 dB	4.1769 dB	75.26s	0.1517s	38
1000	4.1965 dB	3.9045 dB	66.14s	0.4981s	29
2000	5.0473 dB	3.7208 dB	62.89s	3.5166s	22
6948	4.2300 dB	3.8771 dB	74.30s	2.2450s	13

Phase 4: Extreme Learning Machine

- Limited Sample Analysis
- Limited Compute Analysis

Fig. 5: Extreme Learning Machine Model [8]

$$f(x) = \sum_{i=1}^{L} \beta_i g(w_i \cdot x + b_i)$$

Eq. 2: General single-hidden layer feedforward network (SLFN) equation

w_i = weight vector from to node i
b_i = bias of node i
β_i = output weight

$$\hat{\beta} = H^{\dagger} T$$

Eq. 3: Moore-Penrose generalized inverse optimization

Samples	Inception RMSE	ELM RMSE	Inception Train Time	ELM Train Time	Inception Epochs
100	2.1208 dB	7.7002 dB	46.2s	0.0095s	50
500	1.7000 dB	5.1000 dB	46.2s	0.0095s	50
1000	1.2963 dB	3.0642 dB	46.42s	0.0181s	29
2000	0.9675 dB	1.7200 dB	62.40s	5.5100s	22
4000	0.5900 dB	0.4771 dB	70.70s	5.6400s	17

The next phase was to push beyond just the baselines and implement an Exteme Learning Machine. This lightweight model relies on a random hidden layer and only trains on the output weights from the hidden layer to the output layer. This simplification removes expensive back propagation and replaces it with a simple linear time fit function called the Moore-Penrose generalized inverse optimization. As we established earlier, for a quick deployment in a new area, the limited number of samples and limited access to heavy compute resources makes large deep learning models difficult to utilize. I perfomed a limited sample training of the Inception and Extreme Learning Machine models to show how RSME and training time compares. Inception runs for about 60 seconds regardless of the sample size because when there are a smaller number of samples, it takes longer to train to reach our set threshold of 5 RSME. ELM fits much faster and also has better performance at all sample levels.

Phase 4: Extreme Learning Machine

% Train	Inc RMSE	ELM RMSE	Inc Train (s)	ELM Fit (s)	Inc Inf (ms)	ELM Inf (ms)
20%	97.0953	37.9405	1.87	0.0333	0.3427	0.2165
40%	85.6834	12.7268	3.74	0.0670	0.3585	0.2259
60%	93.3388	5.8723	5.61	0.0977	0.3788	0.2295
80%	28.9217	5.1637	7.49	0.1520	0.4784	0.2551

Specific Flight: Dataset_24_PawPrints_5G_30m_Flight_2.csv

% Train	Inc RMSE	ELM RMSE	Inc Train (s)	ELM Fit (s)	Inc Inf (ms)	ELM Inf (ms)
20%	92.4061	29.9600	4.44	0.0873	0.3370	0.2190
40%	78.4127	11.0304	8.62	1.0688	0.3503	0.2304
60%	67.9661	4.9897	12.84	1.1330	0.3556	0.2257
80%	27.0395	4.7115	17.05	2.0811	0.3860	0.2411

All Flights Averaged

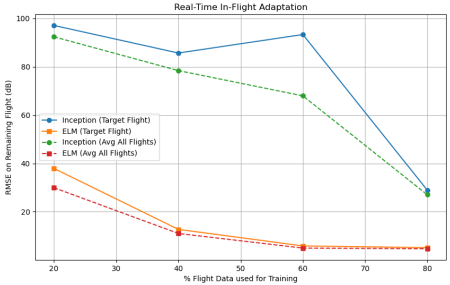


Fig 9: Real-Time In-Flight Learning

VIRGINIA TECH

Wireless @ Virginia Tech

Phase 4: Extreme Learning Machine

% Train	Inc RMSE	ELM RMSE	Inc Time (s)	ELM (s)	Inc Acc	ELM Acc
20%	0.14011	0.11466	1.57	0.0072	0.7457	0.5387
40%	0.12000	0.10	3.14	0.0144	0.7500	0.6000
60%	0.11466	0.0916	4.71	0.0072	0.7576	0.6500
80%	0.0917	0.0917	6.28	0.0156	0.7576	0.6500

Specific Flight Dataset_24_FromPoint_20_OtherFlight_2.csv

% Train	Inc RMSE	ELM RMSE	Inc Time (s)	ELM (s)	Inc Acc	ELM Acc
20%	0.1463	0.0968	1.57	0.0072	0.7576	0.7576
40%	0.1463	0.0968	3.14	0.0144	0.7576	0.7576
60%	0.1463	0.0917	4.71	0.0072	0.7576	0.7576
80%	0.1463	0.0917	6.28	0.0156	0.7576	0.7576

All Flights Averaged

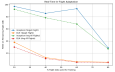


Fig 9: Real-Time In-Flight Learning

To emulate a real-world scenario, I trained the models at specific points throughout each individual flight. At 20%, 40%, 60%, and 80% both models would train and be tested on the rest of the unseen flight. This proved highly effective in showing the usefulness of the ELM model under very limited sample sizes, and the performance was reasonable for the extrapolated data. The graph on the right shows how as the flight progresses, ELM is much faster at learning than Inception, and for a single flight, is almost always more accurate.

Discussion & Conclusion

- What if we had a much larger sample size from roughly the same time?
- At what point does a deep learning model surpass a light wieght model?
- What useful metrics could be collected in a real-world test?

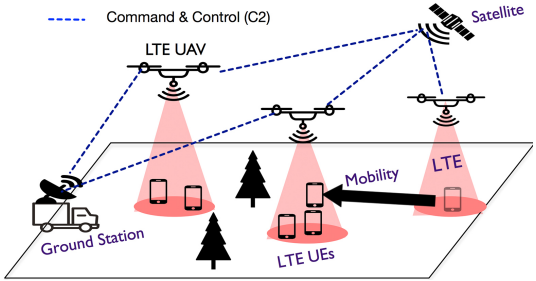
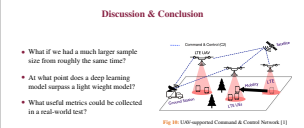


Fig 10: UAV-supported Command & Control Network [1]



From this project, a number of different questions come up. What if we had a massively large dataset? When will deep learning beat extreme learning? What could we get out of a real-world test? These models were put together relatively quickly as a kind of proof of concept, and more fine tuning may yield better results. Real world scenarios are also more complex with more than just one UAV and one Base Station. Integrating this work into an actual deployment scheme would require more effort. Overall, this project shows how edge machine learning can be useful in specific scenarios where UAVs may be used for supporting a wireless network.

Future Work

- Further Model Analysis
- AERPAW Experimentation
- Iterative Design

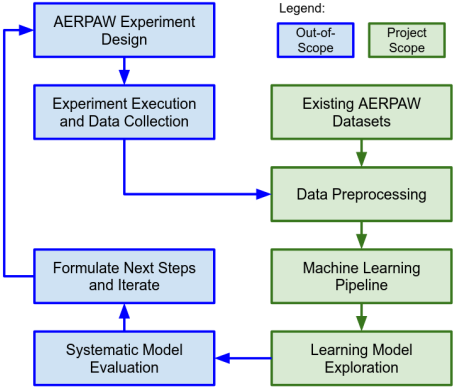


Fig 11: Project Layout Flowchart

Future Work

- Further Model Analysis
- AERPAW Experimentation
- Iterative Design

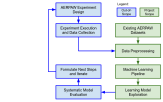


Fig 14: Project Layout Flowchart

This project not only serves a standalone investigation into deep learning and lightweight learning models for UAV radio map modeling, but it will also enable future research into this area. This project did not have time to perform a systematic analysis over several different learning models, or create an actual AERPAW experiment. But this project does give Shadab and Dr. Haihan a better understanding of the research space and how they can use their skills to make an impact in this area.

Questions